

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

زمستان ۱۴۰۱

آموزش مدل‌ها

Training Models

رگرسیون خطی

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = h_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta} \cdot \mathbf{x} = \boldsymbol{\theta}^T \mathbf{x}$$

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^{\dagger} \mathbf{y}$$

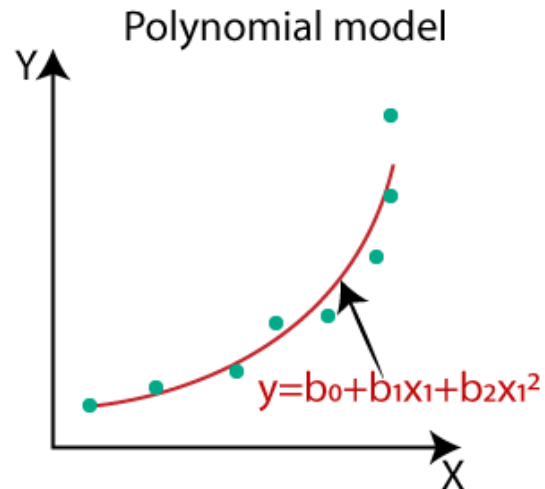
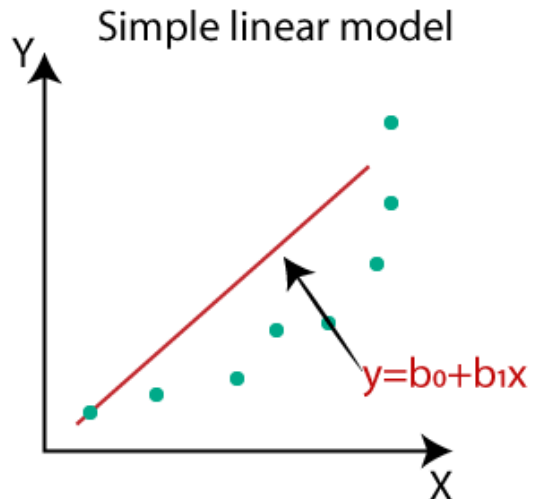
$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

Algorithm	Large m	Out-of-core support	Large n	Hyperparams	Scaling required	Scikit-Learn
Normal equation	Fast	No	Slow	0	No	N/A
SVD	Fast	No	Slow	0	No	LinearRegression
Batch GD	Slow	No	Fast	2	Yes	N/A
Stochastic GD	Fast	Yes	Fast	≥ 2	Yes	SGDRegressor
Mini-batch GD	Fast	Yes	Fast	≥ 2	Yes	N/A

رگرسیون چند جمله‌ای (Polynomial)

- ممکن است داده پیچیده‌تر از یک خط مستقیم باشد
- خوشبختانه می‌توان از یک مدل خطی برای رگرسیون غیرخطی استفاده کرد
 - یک راه ساده این است که ویژگی‌های جدید غیرخطی از ویژگی‌های اولیه استخراج کرد و یک رابطه خطی بر اساس آنها آموزش داد
- به عنوان مثال، توان‌های ویژگی‌ها که در این صورت رگرسیون چند جمله‌ای نامیده می‌شود



رگرسیون چند جمله‌ای

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)

>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929,  0.56664654])

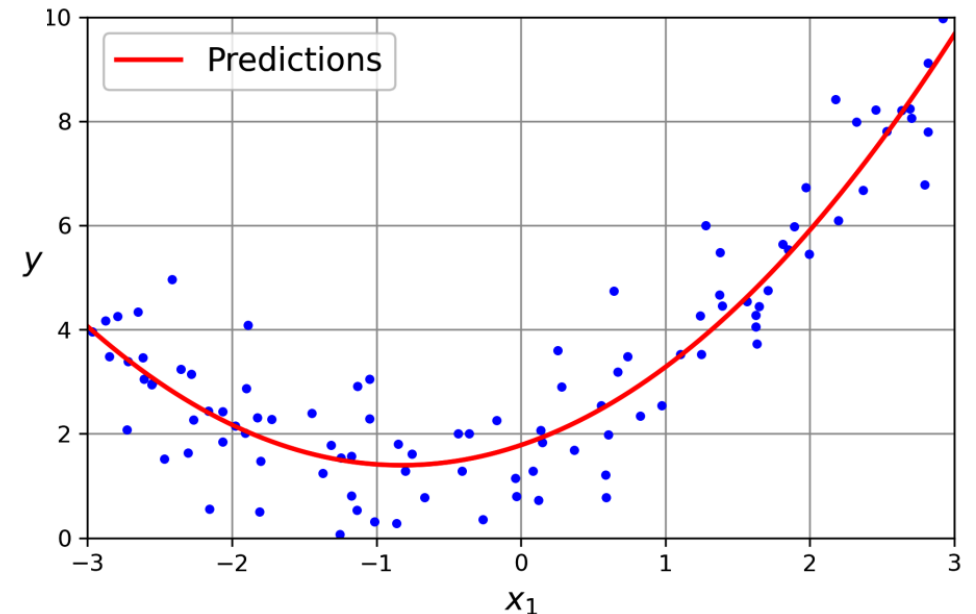
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893,  0.56456263]]))
```

$$\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$$

$$y = 0.5x_1^2 + 1.0x_1 + 2.0 + \text{Gaussian noise}$$

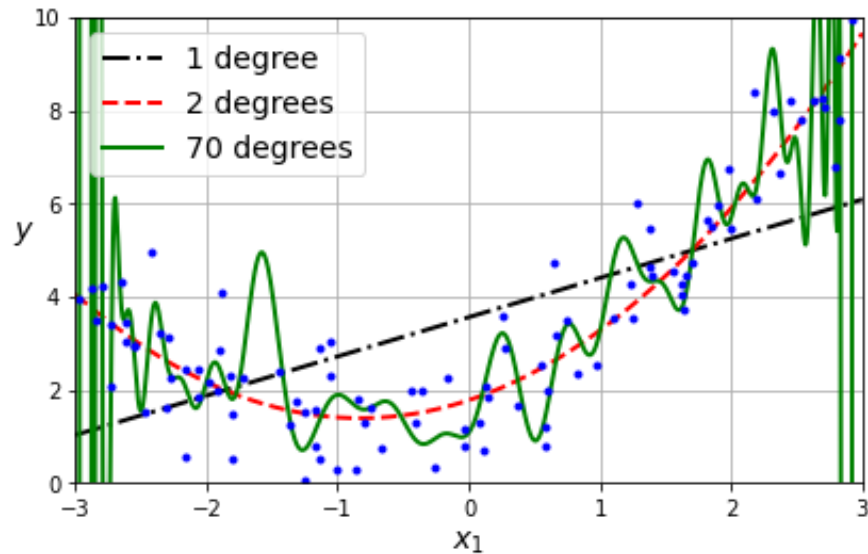
- اگر چند ویژگی داشته باشیم، ترکیب آنها هم ساخته می‌شود که در مجموع $(n + d)!/n!d!$ ویژگی می‌شود

- مثلاً a^2b و ab^2 , ab , b^3 , b^2 , a^3 , a^2



انتخاب مرتبه

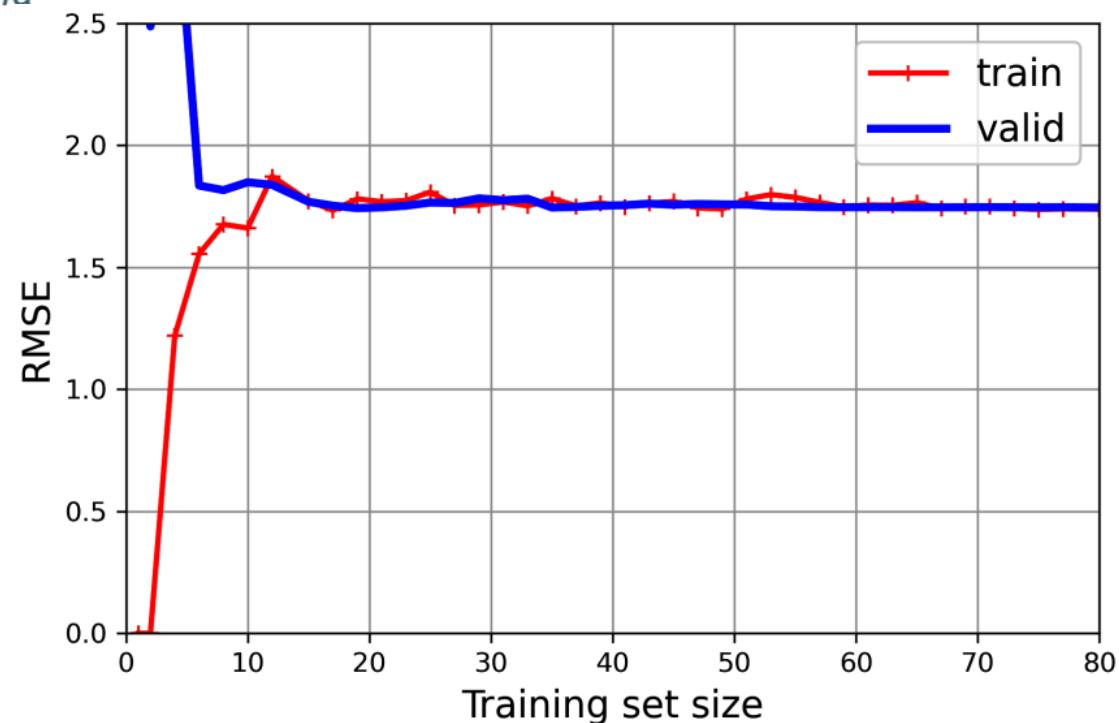
- چگونه مرتبه را انتخاب کنیم؟
- چندجمله‌ای مرتبه بالا می‌تواند به خوبی به دادگان آموزشی fit شود اما لزوماً تعمیم‌دهی خوبی نخواهد داشت
- ممکن است مسئله واقعاً چندجمله‌ای نباشد که البته با بسط تیلور قابل تبدیل است



```
from sklearn.model_selection import learning_curve
```

```
train_sizes, train_scores, valid_scores = learning_curve(  
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,  
    scoring="neg_root_mean_squared_error")  
train_errors = -train_scores.mean(axis=1)  
valid_errors = -valid_scores.mean(axis=1)
```

```
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")  
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")  
[...] # beautify the figure: add labels, axis, grid, and legend  
plt.show()
```

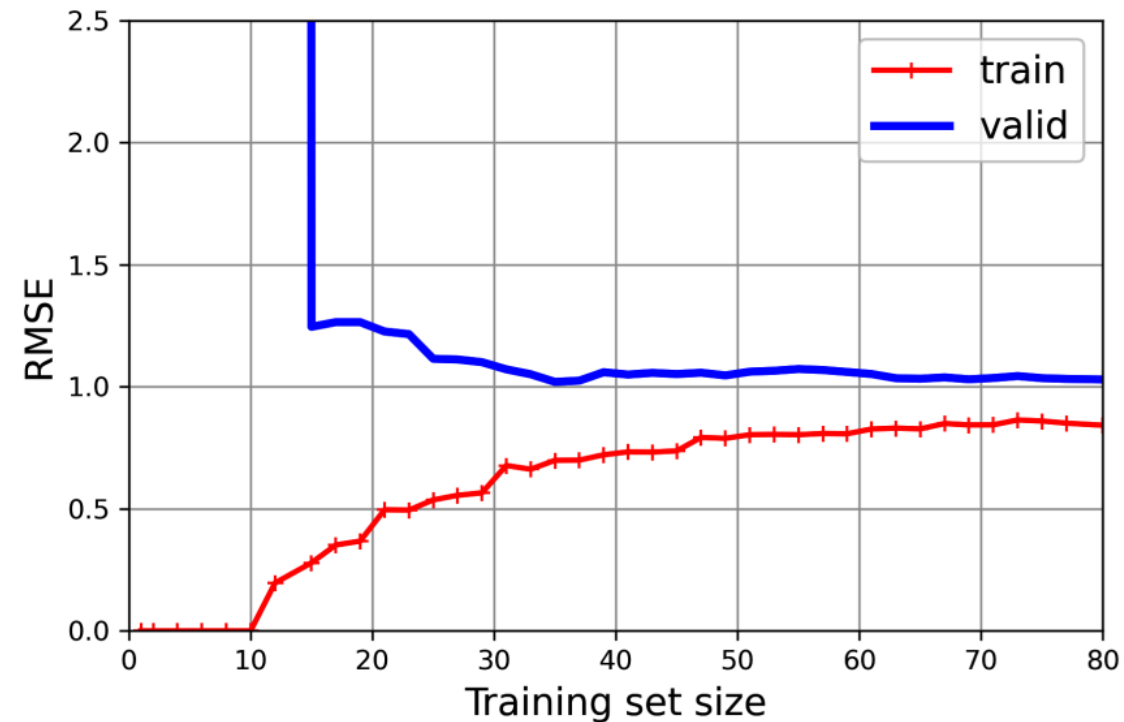



```
from sklearn.pipeline import make_pipeline
```

```
polynomial_regression = make_pipeline(  
    PolynomialFeatures(degree=10, include_bias=False),  
    LinearRegression())
```

```
train_sizes, train_scores, valid_scores = learning_curve(  
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,  
    scoring="neg_root_mean_squared_error")
```

```
[...] # same as earlier
```



مدل‌های چندجمله‌ای منظم‌شده (Regularized)

- یک راه خوب برای کاهش بیش‌برازش، محدود کردن و منظم‌سازی مدل است
 - هرچه درجه آزادی کمتری داشته باشد، بیش‌برازش داده‌ها سخت‌تر خواهد بود
 - یک راه ساده برای منظم‌سازی یک مدل چندجمله‌ای، کاهش درجه چندجمله‌ای است
 - می‌توان وزن‌های مدل را محدود کرد

Ridge Regression

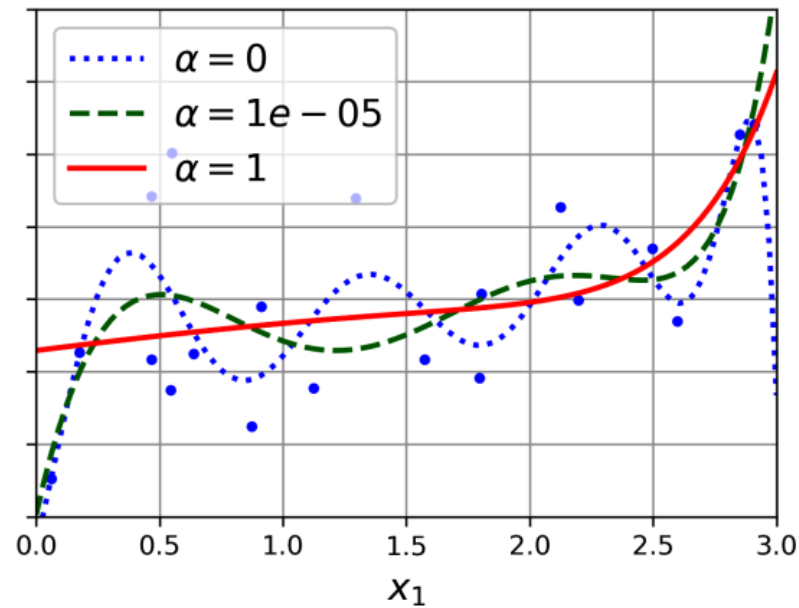
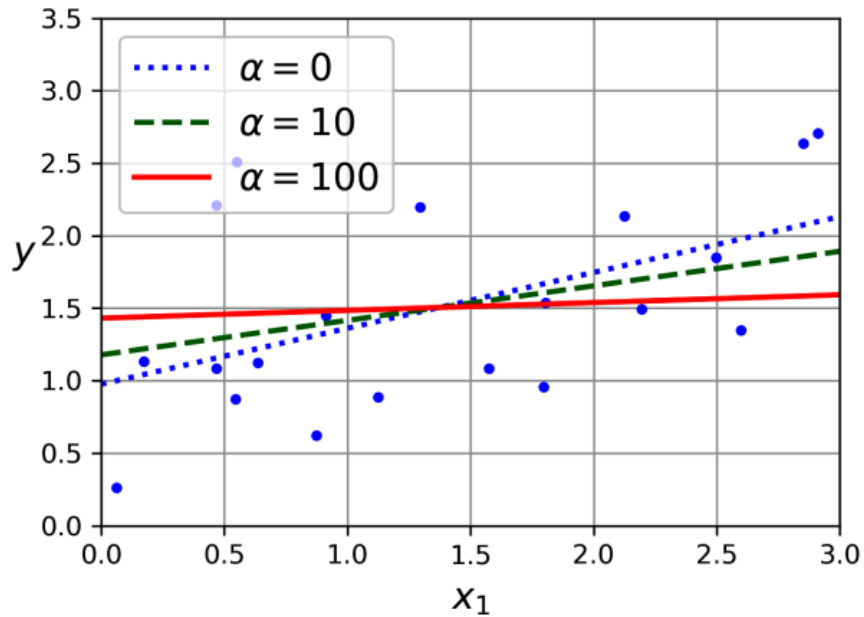
- یک راه خوب برای کاهش بیش‌برازش، محدود کردن و منظم‌سازی مدل است
- در رگرسیون Ridge، یک عبارت منظم‌سازی مرتبط با اندازه وزن‌ها به MSE اضافه می‌شود

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$$

- تلاش می‌کند با وزن‌های کوچک (تابع ساده‌تر) مسئله را حل کند
- این عبارت فقط در زمان آموزش اضافه می‌شود و برای ارزیابی عملکرد مدل حذف می‌شود
- معمولاً روی θ_0 محدودیتی قرار داده نمی‌شود
- مهم است که ویژگی‌ها مقیاس شده باشند تا محدودیت اعمال شده نسبت به مقیاس ویژگی‌ها حساس نباشد

Ridge Regression

- مثالی از رگرسیون خطی و رگرسیون چندجمله‌ای مرتبه ۱۰
- با افزایش α ، پیش‌بینی‌ها مسطح‌تر می‌شوند (واریانس مدل کاهش می‌یابد)



Ridge Regression

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$$

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{A})^{-1} \mathbf{X}^T \mathbf{y}$$

- این مسئله بهینه‌سازی هم مرتبه ۲ است و پاسخ بسته دارد

- ماتریس $\mathbf{A}$ شبیه به ماتریس همانی $(n+1) \times (n+1)$ است که مولفه اول آن صفر است

```
>>> from sklearn.linear_model import Ridge
>>> ridge_reg = Ridge(alpha=0.1, solver="cholesky")
>>> ridge_reg.fit(X, y)
>>> ridge_reg.predict([[1.5]])
array([[1.55325833]])
```

- با SGD هم به سادگی قابل حل است

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) - 2\alpha \boldsymbol{\theta} / m$$

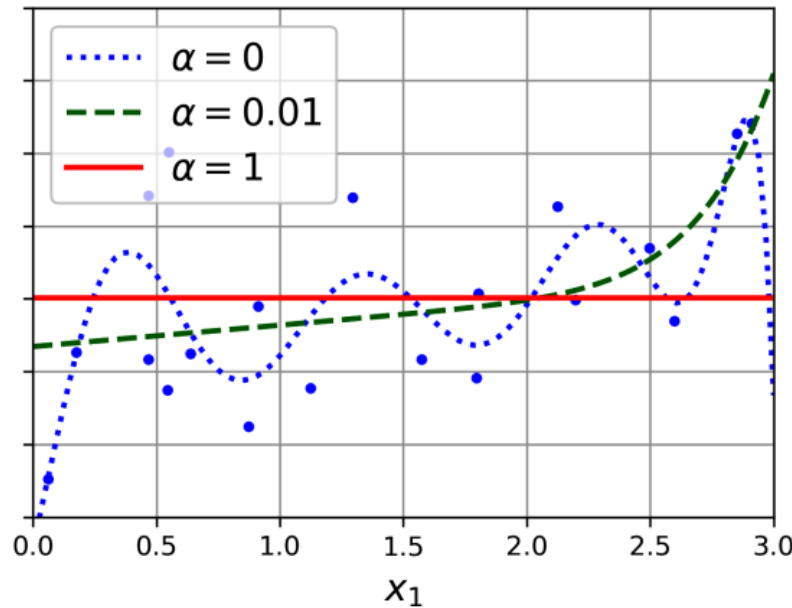
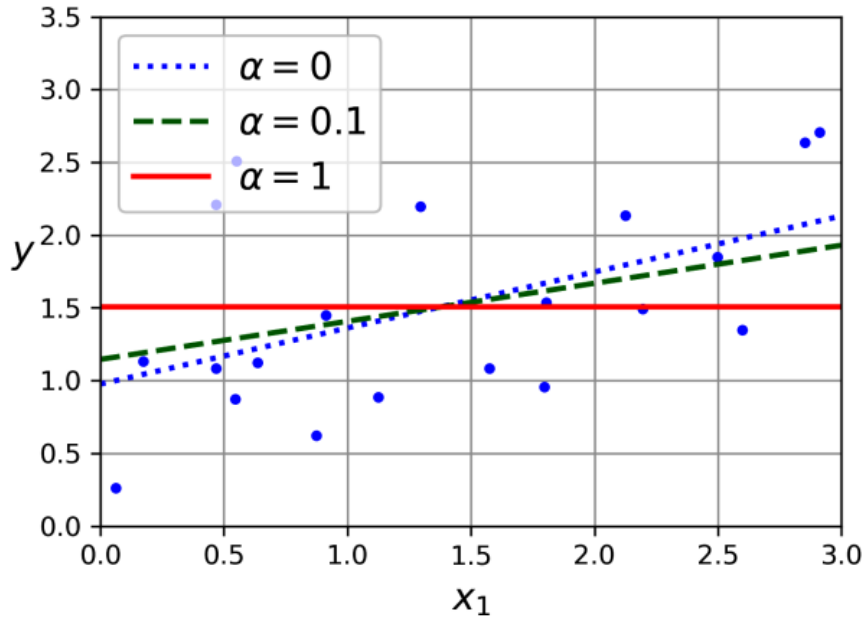
```
>>> sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
...                          max_iter=1000, eta0=0.01, random_state=42)
...
>>> sgd_reg.fit(X, y.ravel()) # y.ravel() because fit() expects 1D targets
>>> sgd_reg.predict([[1.5]])
array([1.55302613])
```

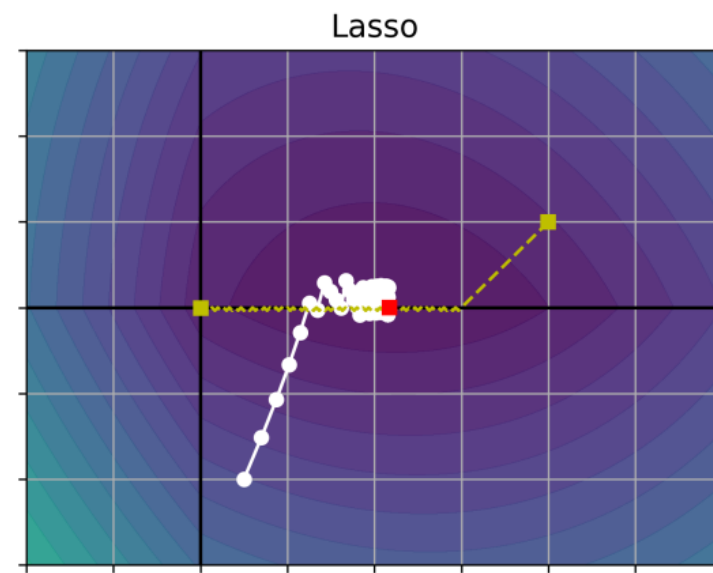
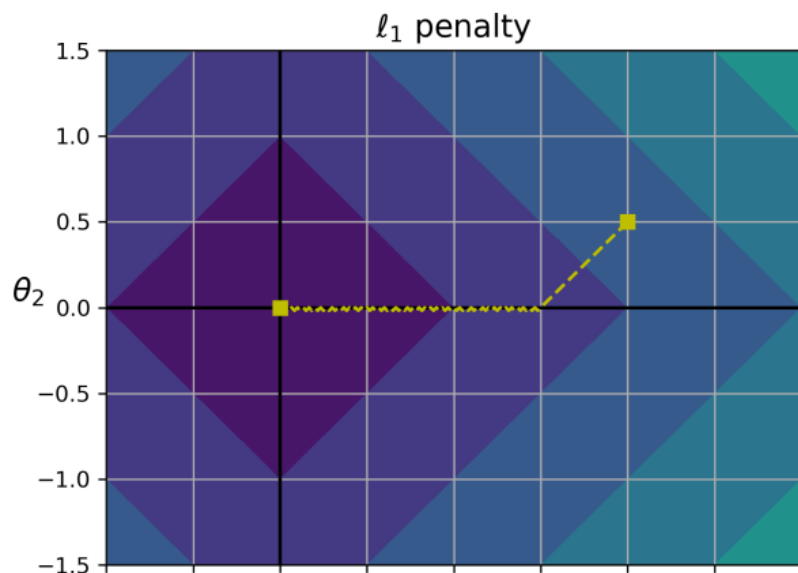
Lasso Regression

- برای منظم‌سازی از نُرم ۱ استفاده می‌کند

- یک مشخصه مهم Lasso این است که وزن‌های مربوط به ویژگی‌های کم‌اهمیت را صفر می‌کند

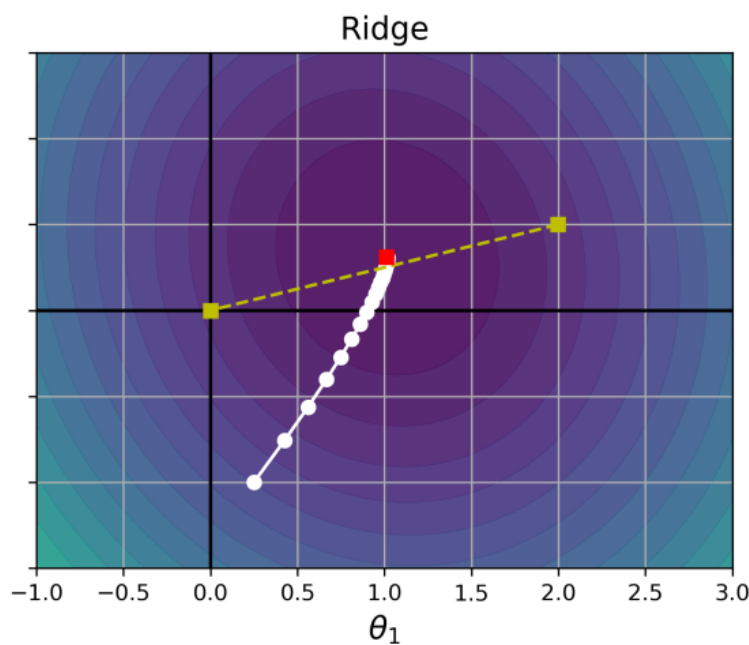
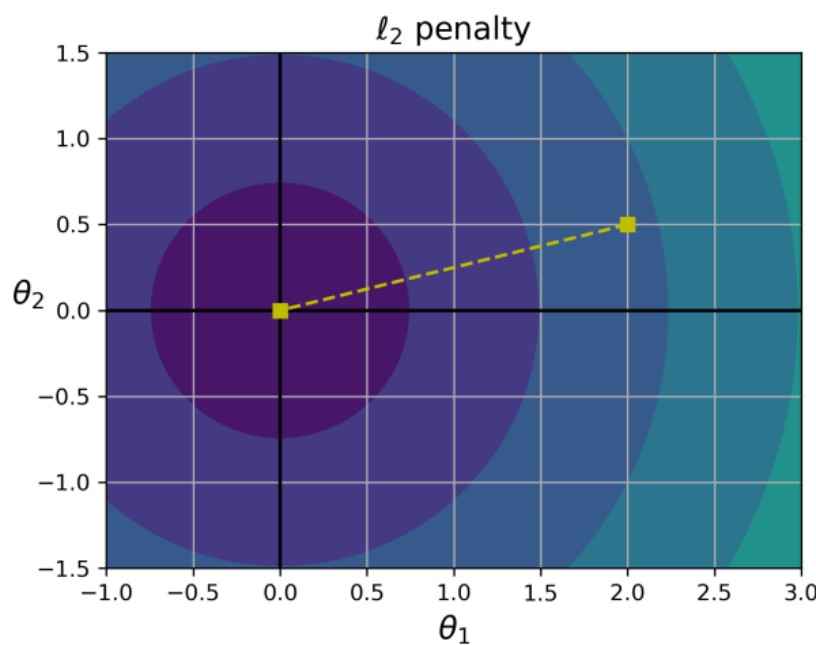
- به نوعی به صورت خودکار انتخاب ویژگی را انجام می‌دهد و مدل حاصل تُنک (sparse) خواهد بود





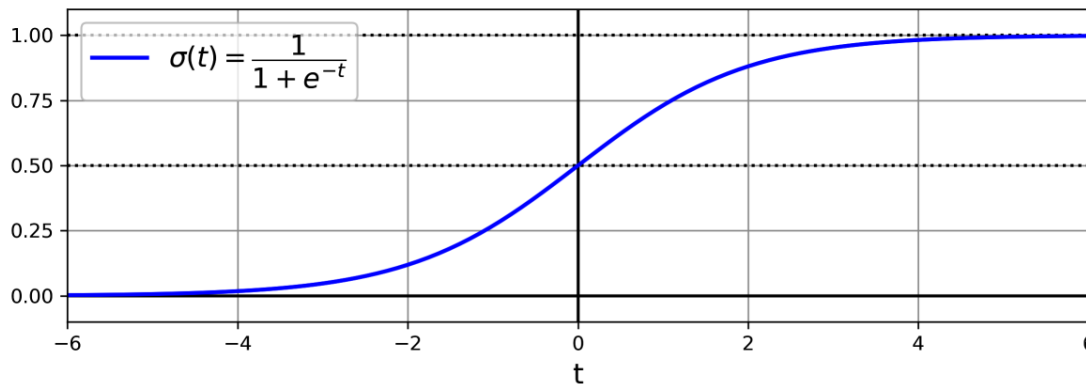
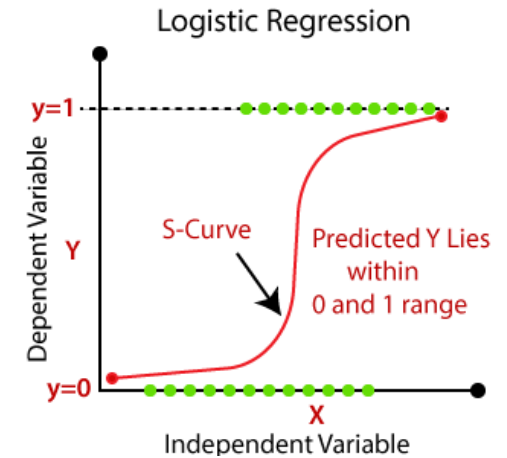
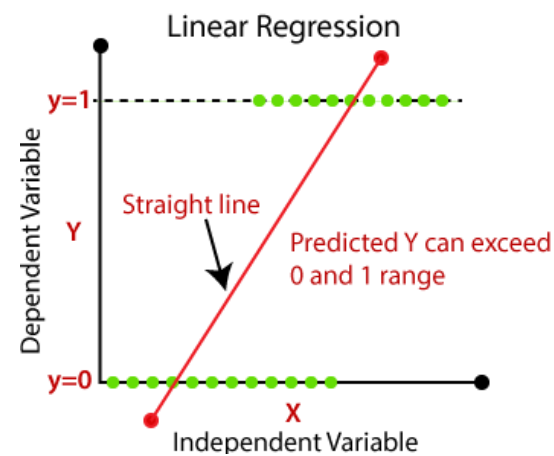
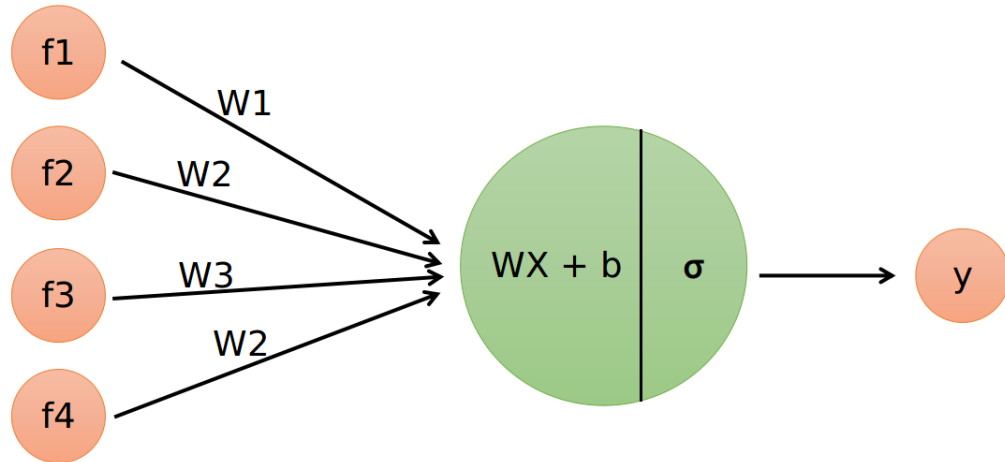
● نقطه بهینه برای هر دو تابع جریمه
 ℓ_1 و ℓ_2 مرکز مختصات است اما
 مسیر بهینه‌سازی آنها از نقطه شروع
 یکسان متفاوت است

- در ℓ_1 ، θ_2 (که مقدار اولیه آن 0.5
 است) سریعتر صفر می‌شود



Logistic Regression

- با محدود کردن خروجی رگرسیون خطی، می‌توان از آن برای پیش‌بینی احتمال و دسته‌بندی استفاده کرد



$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x})$$

Logistic Regression

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x})$$

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

$$\frac{1}{1 + \exp(-t)} = p \Rightarrow 1 = p + p \exp(-t) \Rightarrow 1 - p = p \exp(-t)$$

$$\Rightarrow \frac{1 - p}{p} = \exp(-t) \Rightarrow t = \log\left(\frac{p}{1 - p}\right) = \text{logit}(p)$$

- با این تابع می‌توان احتمال تعلق $\mathbf{x}$ به کلاس ۱ را محاسبه کرد

- توجه شود که $\sigma(t) < 0.5$ در $t < 0$ اتفاق می‌افتد

- $\theta^T \mathbf{x} = 0$ معادل با احتمال 0.5 است و مرز تصمیم را مشخص می‌کند

- به امتیاز t گفته می‌شود logit

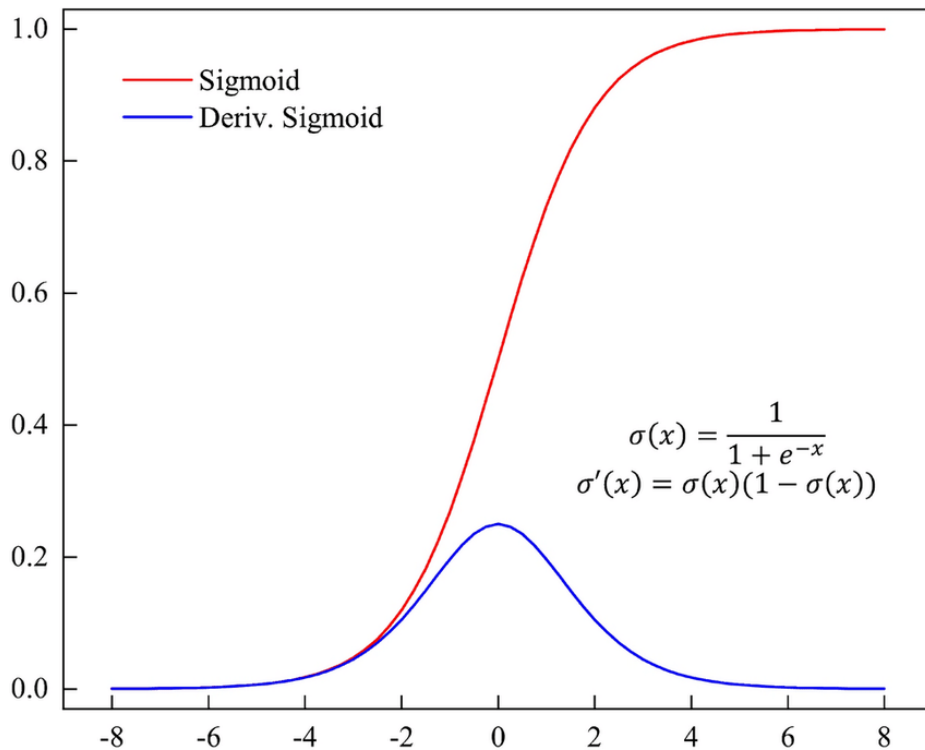
Logistic Regression

$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

- از تابع هزینه Cross Entropy استفاده می‌کنیم

- با توجه به اشباع شدن تابع Sigmoid و کوچک بودن گرادیان آن در بسیاری از مقادیر، CE مناسب‌تر از تابع هزینه MSE است

- مثال عددی: اگر $o = 5.6$ و $y = 0$:



$$c = (y - \sigma(o))^2$$

$$\frac{dc}{do} = -2(y - \sigma(o))\sigma(o)(1 - \sigma(o)) = 0.0073$$

$$c = -\log(1 - \sigma(o))$$

$$\frac{dc}{do} = -\frac{-\sigma(o)(1 - \sigma(o))}{1 - \sigma(o)} = \sigma(o) = 0.9963$$

Logistic Regression

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

- پاسخ بسته‌ای برای یافتن مینیمم این تابع محاسبه نشده است
- اما این تابع محدب است، بنابراین با استفاده از GD می‌توان نقطه بهینه سراسری آن را بدست آورد

$$\frac{\partial}{\partial \theta_j} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \left(\sigma(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- برای هر نمونه، خطای پیش‌بینی را محاسبه می‌کند و آن را در مقدار ویژگی x_j ضرب می‌کند

مرزهای تصمیم

- برای شبیه‌سازی از مجموعه داده iris استفاده می‌کنیم
- این مجموعه داده شامل ۴ ویژگی (عرض و ارتفاع کاسبرگ و گلبرگ) از ۱۵۰ گل زنبق از ۳ کلاس مختلف است



```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> list(iris)
>>> iris.data.head(3)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
>>> iris.target.head(3) # note that the instances are not shuffled
0    0
1    0
2    0
```

مرزهای تصمیم

- فرض کنید می‌خواهیم یک دسته‌بند برای تشخیص گل‌های virginica تنها با استفاده از عرض گلبرگ آموزش بدهیم

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

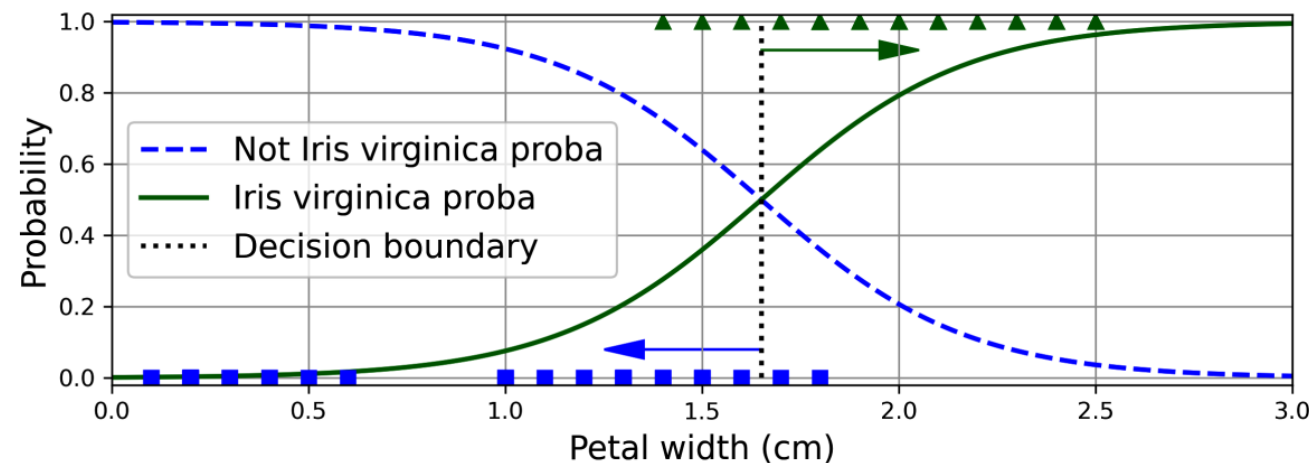
```
X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

```
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]
```

```
>>> decision_boundary
1.6516516516516517
```

```
>>> log_reg.predict([[1.7], [1.5]])
array([ True, False])
```

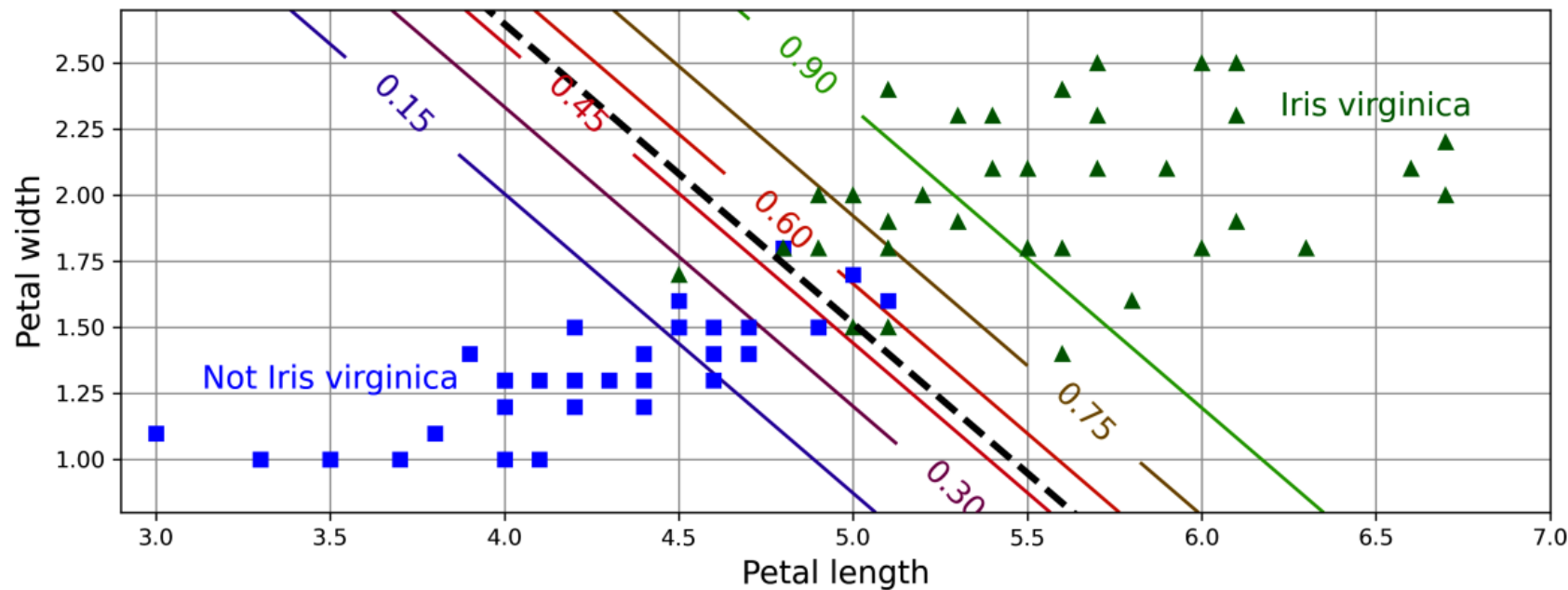


مرزهای تصمیم

- حال همان مسئله را با استفاده از دو ویژگی بررسی می کنیم

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x})$$

- مرز تصمیم به چه صورتی است؟



Softmax Regression

- مدل logistic regression می تواند به سادگی برای دسته بندی چند کلاسه تعمیم داده شود
 - بدون نیاز به آموزش و ترکیب چندین دسته بند باینری

- برای هر کلاس، یک تابع امتیاز $s_k(\mathbf{x})$ آموزش داده می شود و سپس با نرمال سازی، یک احتمال برای هر کلاس تخمین زده می شود

$$s_k(\mathbf{x}) = (\boldsymbol{\theta}^{(k)})^\top \mathbf{x}$$

- هر کلاس بردار پارامترهای خاص $\boldsymbol{\theta}^{(k)}$ خودش را دارد
- مجموع احتمالات با ۱ است

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

$$\hat{y} = \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k = \operatorname{argmax}_k s_k(\mathbf{x}) = \operatorname{argmax}_k ((\boldsymbol{\theta}^{(k)})^\top \mathbf{x})$$

Softmax Regression

- تابع هزینه مناسب برای این مدل، تابع Cross Entropy است
- برای حالت دو کلاسه دقیقا مشابه با BCE است

$$J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

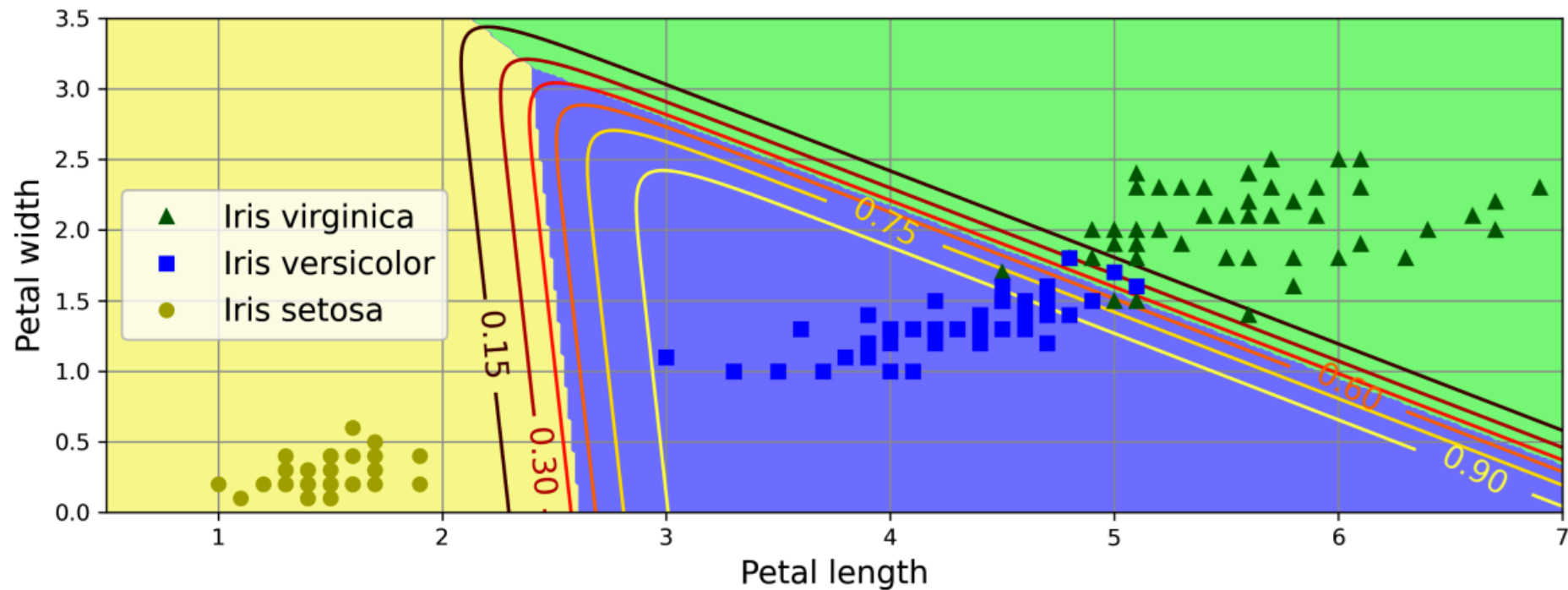
$$\nabla_{\Theta^{(k)}} J(\Theta) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

- بردار گرادیان برای وزنهای کلاس k :


```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values  
y = iris["target"]  
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

```
softmax_reg = LogisticRegression(C=30, random_state=42)  
softmax_reg.fit(X_train, y_train)
```

```
>>> softmax_reg.predict([[5, 2]])  
array([2])  
>>> softmax_reg.predict_proba([[5, 2]]).round(2)  
array([[0.  , 0.04, 0.96]])
```



مدل‌های خطی

- در این فصل، روش‌های مختلفی برای آموزش مدل‌های خطی (هم برای رگرسیون و هم برای دسته‌بندی) بررسی شد

- برای رگرسیون خطی پاسخ بسته محاسبه شد $\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^+ \mathbf{y}$

- استفاده از گرادیان کاهشی (و نسخه‌های stochastic آن) بررسی شد $\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$

- با استفاده از طراحی ویژگی‌های غیرخطی، می‌توان به توابع غیرخطی با استفاده از مدل‌های خطی رسید

- افزودن عبارت‌های برای جریمه کردن مقادیر نامناسب برای وزن‌ها بررسی شد $J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$

- رگرسیون‌های Logistic و Softmax برای دسته‌بندی خطی معرفی شد $\hat{p} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})$

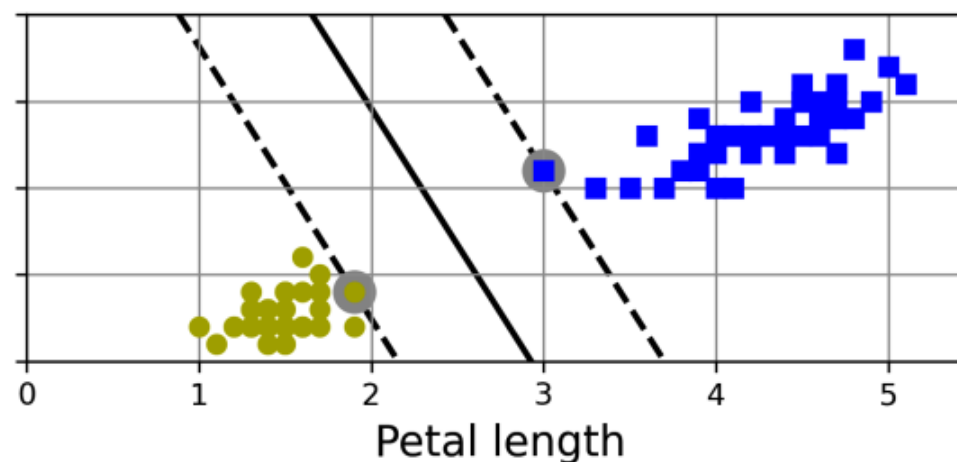
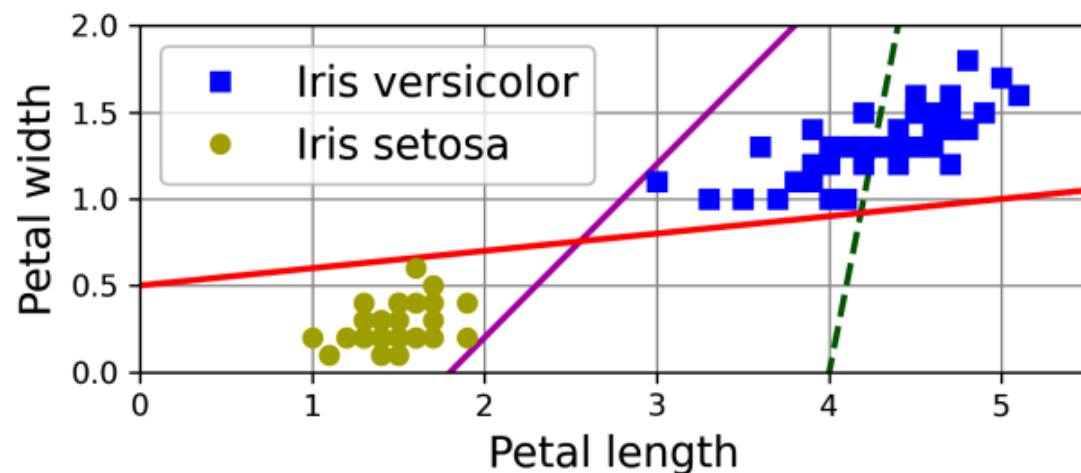
$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

ماشین‌های بردار پشتیبان

Support Vector Machines

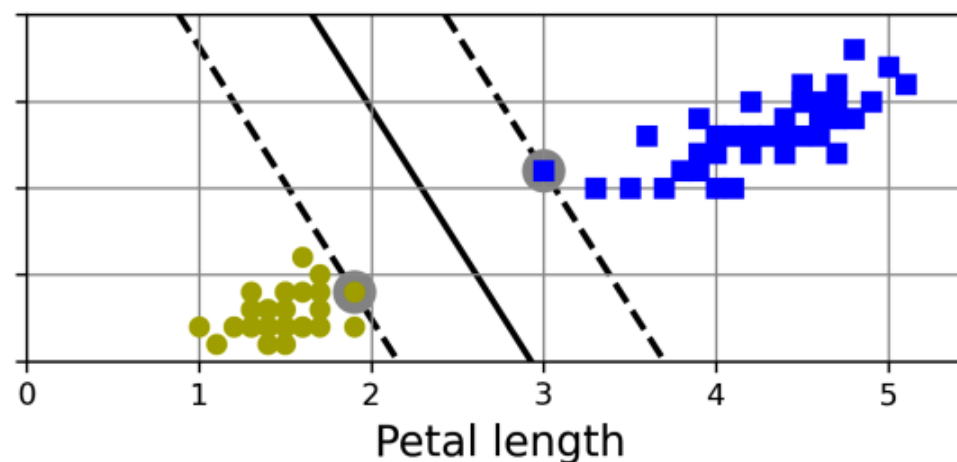
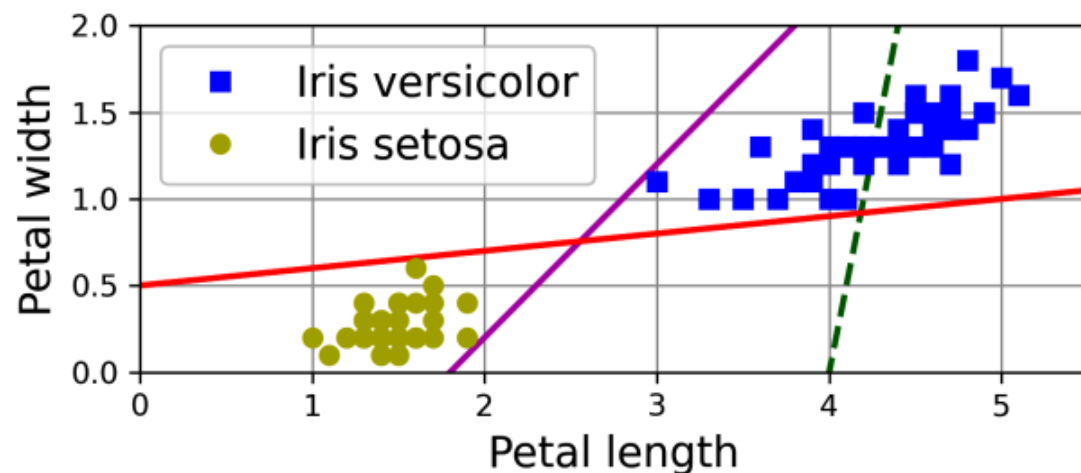
SVM خطی برای دسته‌بندی

- کدام مرز تصمیم مناسب‌تر است؟
- داده‌های این مسئله به صورت خطی جداپذیر است (linearly separable) و هر دو خط قرمز و بنفش روی داده‌های آموزشی دقت کامل دارند
- ایده اساسی SVM این است که مرز حاصل، حداکثر فاصله را با نزدیک‌ترین نمونه‌های آموزشی داشته باشد
- دسته‌بندی با حاشیه بزرگ (large margin) نامیده می‌شود



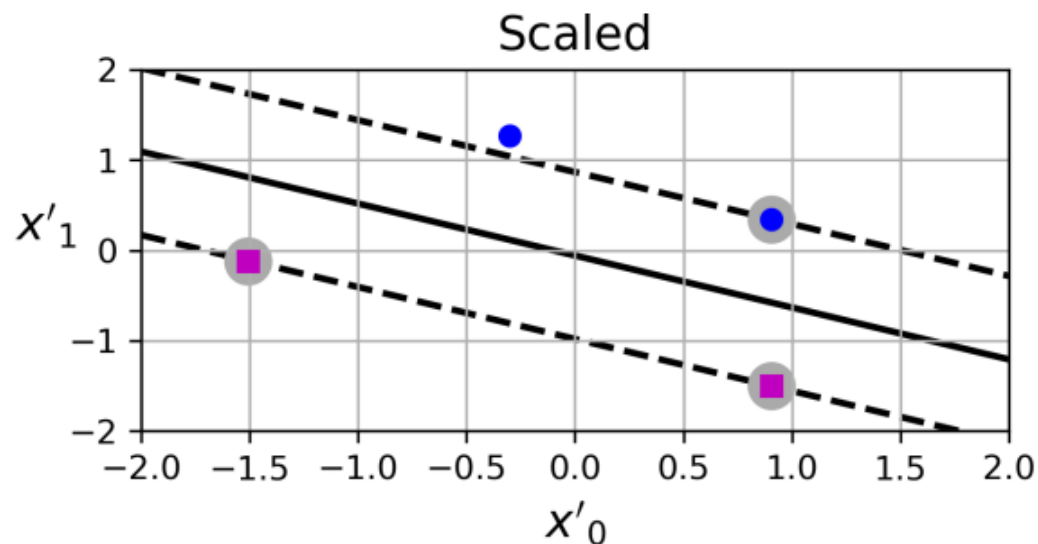
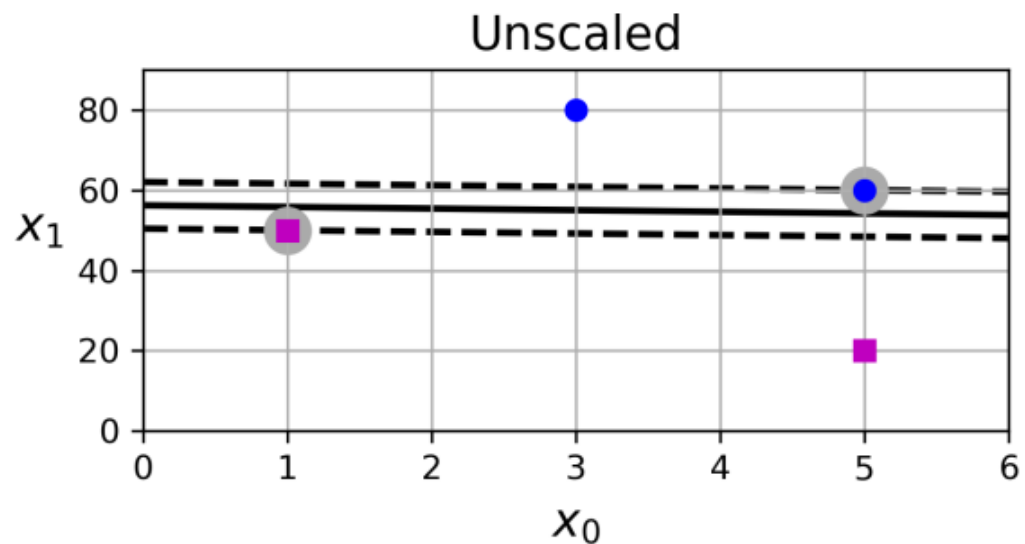
SVM خطی برای دسته‌بندی

- به نمونه‌هایی که کمترین فاصله تا مرز را دارند، بردار پشتیبان (support vector) گفته می‌شود
- افزودن نمونه‌هایی به مجموعه داده که خارج از این حاشیه باشند ("off the street")، تاثیری در مرز تصمیم ندارند



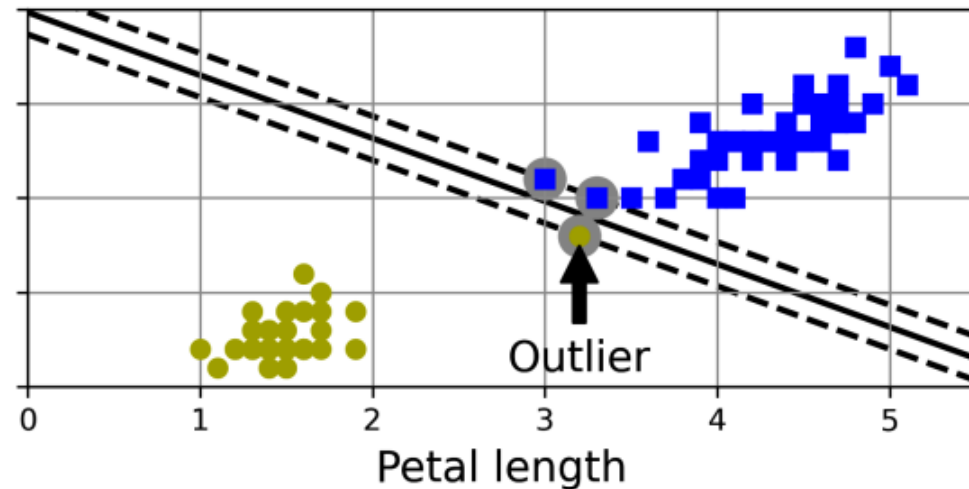
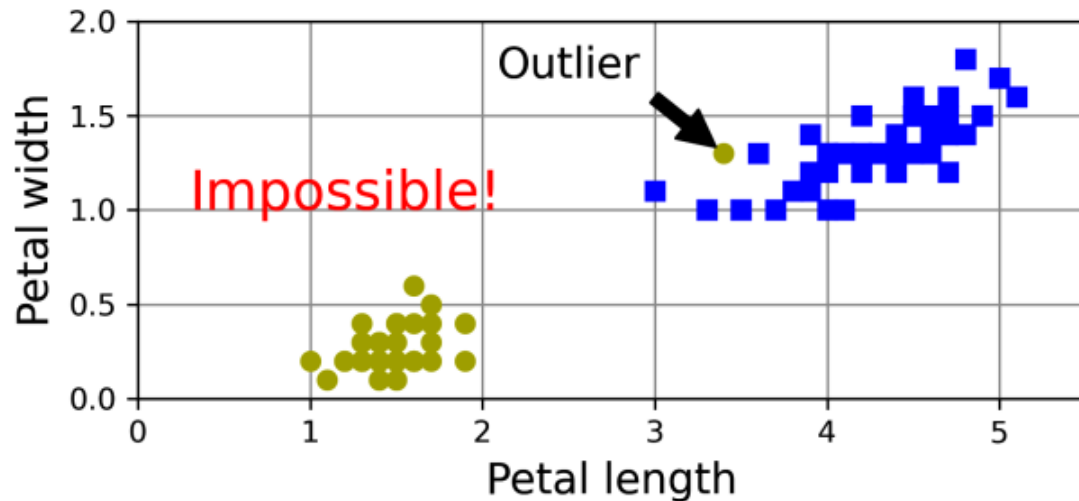
SVM خطی برای دسته‌بندی

- آیا مقیاس ویژگی‌ها در تعیین بردارهای پشتیبان موثر است؟
 - بله! استانداردسازی ویژگی‌ها مهم است



حاشیه نرم (Soft Margin)

- در حالت حاشیه سخت (hard)، حساست نسبت به داده‌های پرت بسیار زیاد است و حتی مسائلی که به صورت خطی جداپذیر نیستند قابل استفاده نیست
- در حاشیه نرم، کمی منعطف‌تر رفتار می‌شود و سعی می‌شود حاشیه تا حد امکان زیاد باشد و نمونه‌هایی که از حاشیه عبور می‌کنند، جریمه ایجاد کنند



حاشیه نرم

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

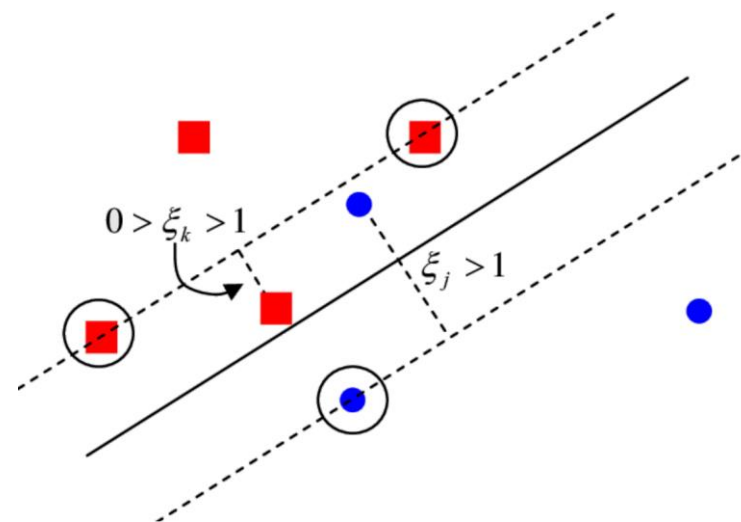
$$\text{s. t. } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \zeta_i$$

$$\text{s. t. } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \zeta_i, \forall i = 1, \dots, m, \zeta_i \geq 0$$

• در حالت **حاشیه سخت**، مسئله زیر حل می شود:

• در حالت **حاشیه نرم**، محدودیت گامسته شده و به مقدار فاصله از حاشیه، به تابع هزینه افزوده می شود



حاشیه نرم

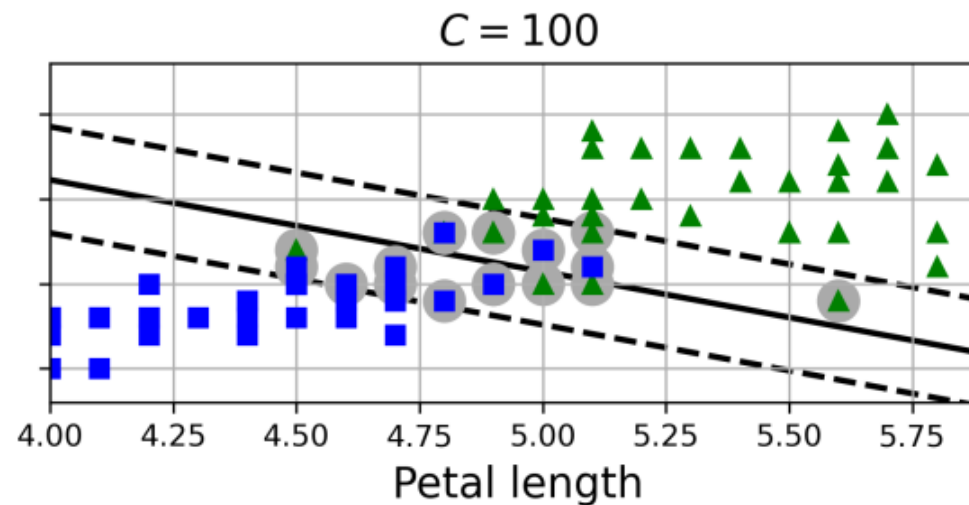
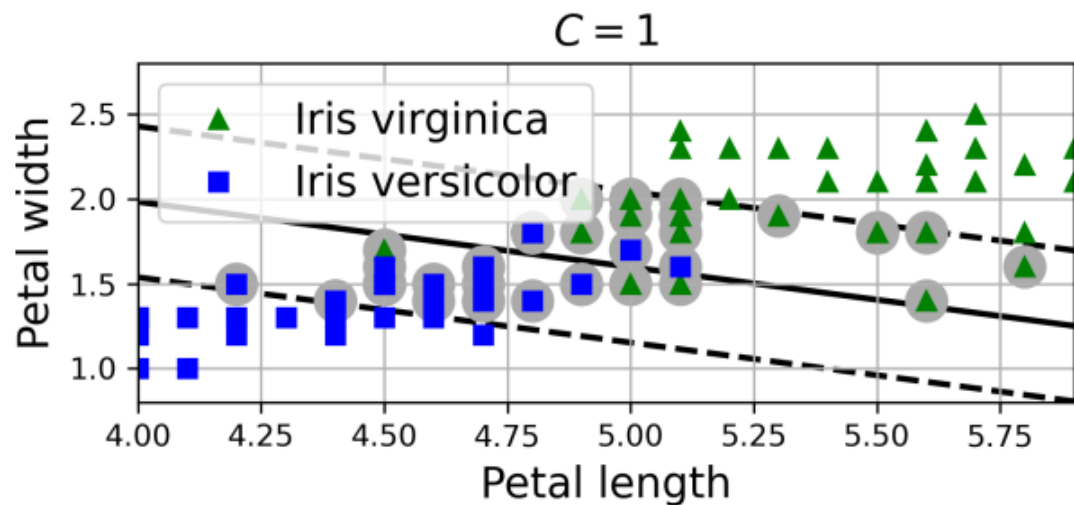
```
from sklearn.svm import LinearSVC
```

```
iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # Iris virginica
```

```
svm_clf = make_pipeline(StandardScaler(),
                        LinearSVC(C=1, random_state=42))
```

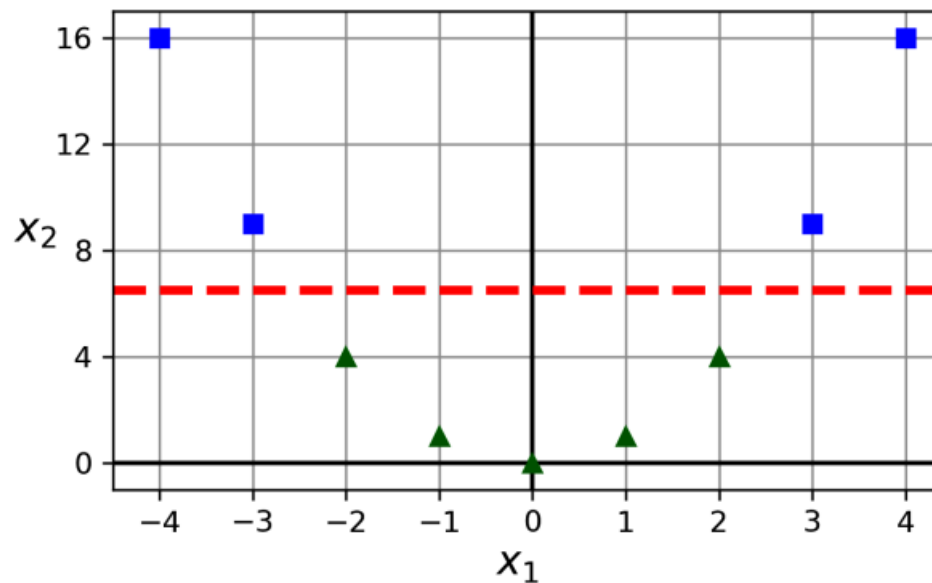
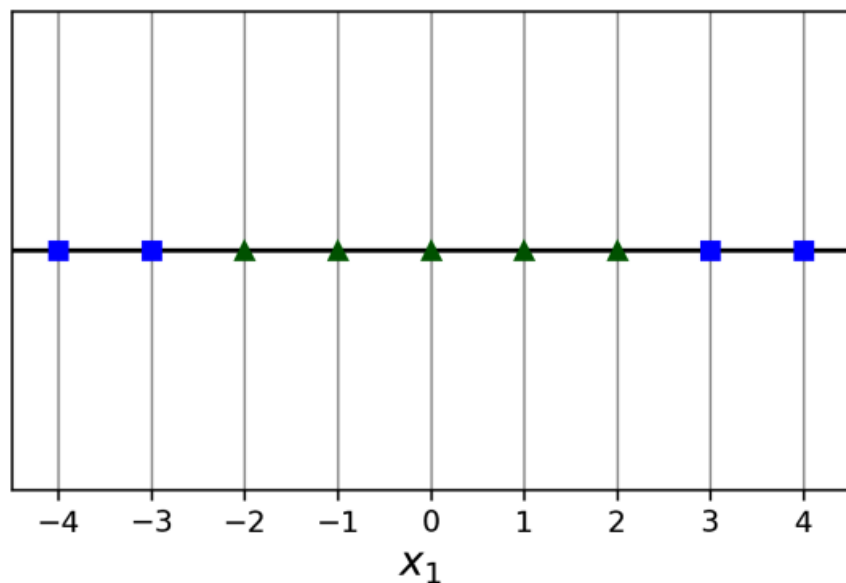
```
svm_clf.fit(X, y)
```

- مدل LinearSVC احتمال محاسبه نمی‌کند بلکه فاصله تا خط تصمیم را محاسبه کرده و بر اساس آن پیش‌بینی می‌کند



SVM غیرخطی برای دسته‌بندی

- برای بسیاری از مجموعه داده‌ها، مدل خطی نمی‌تواند کارایی لازم را داشته باشد
 - یک رویکرد افزودن ویژگی‌های غیرخطی (مانند چند جمله‌ای) و استفاده از مدل خطی است
- $x_2 = (x_1)^2$ -



```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures
```

```
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
```

```
polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
)
polynomial_svm_clf.fit(X, y)
```

